ReactJS

Redux:

What is Redux?

- Redux is a state management library often used with React to handle and centralize application state in a predictable way.
- Redux is a central store for your application state. Instead of keeping state scattered across many components in React, Redux keeps everything in one place called the store.

So instead of:

- Component A managing some state
- Component B managing related state
- Passing props around everywhere

You get:

- One global state container
- Any component can access or update it

Problems without Redux

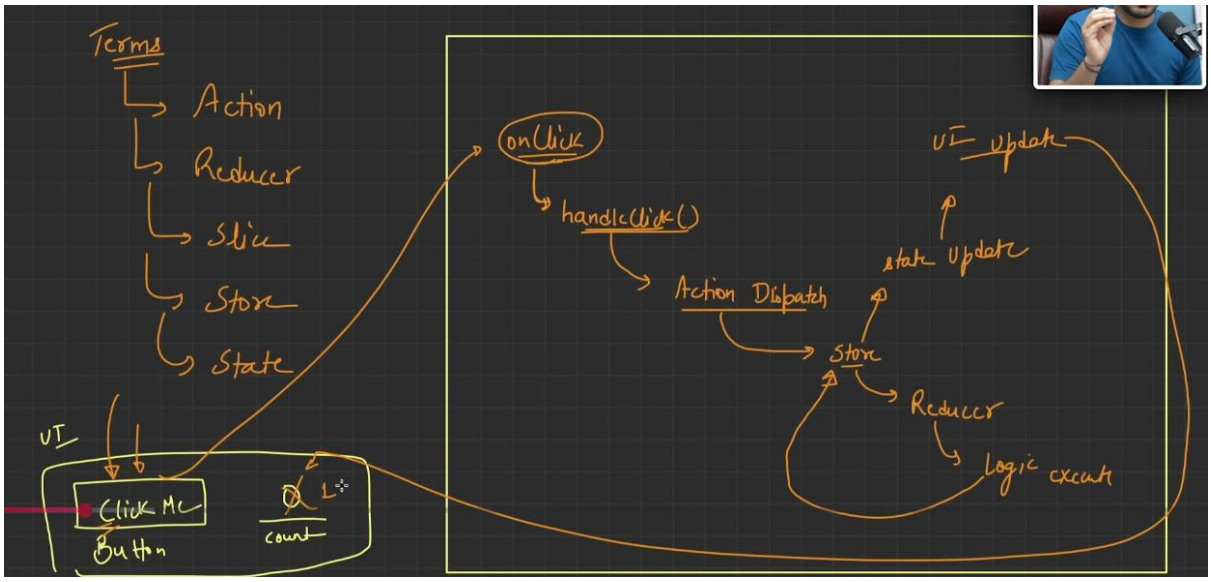
- Prop drilling (passing props through many layers)
- Hard to share state between distant components
- Multiple components updating same state in inconsistent ways
- Debugging becomes difficult
- State logic scattered everywhere

Core Concepts of Redux: Redux is built on a few simple ideas:

1. Action
2. Reducer
3. Slice
4. Store
5. State

Component → Action → Reducer/Slice → Store → State → Component updates

Understanding the flow using the flow diagaram: here we are clicking on the button which has the onClick event which is triggering the function handleClick() which will increase the value of count.



Let's understand each concept of redux in details one by one:

What is an Action?

- An action is a plain object describing an event.
- A description of *something that happened* in the app, along with any extra data required to process it.
- Action = Event(type-manadatory) + Data(additional info- optional) needed to handle that event

What is Slice?

- It is a combination of state + reducers + actions for a specific part of the application.
- It represents one "section" (or "feature") of your global Redux store.
- A slice is a feature-based module in Redux that contains state, reducers, and automatically generated actions for a specific part of the store.
- Slice contains the following things: initial state (Starting data for that feature), reducers (Functions that describe how state changes) and name (Used to prefix action types).

What is Reducer?

- A function that decides how the state should change based on an action.
- A reducer must NOT modify the original state directly. Instead, it must return a new copy of state.
- UI → dispatch(action) → reducer → new state → UI updates
- A reducer is a pure function that takes the current state and an action, and returns the next state of the application.

What is Store?

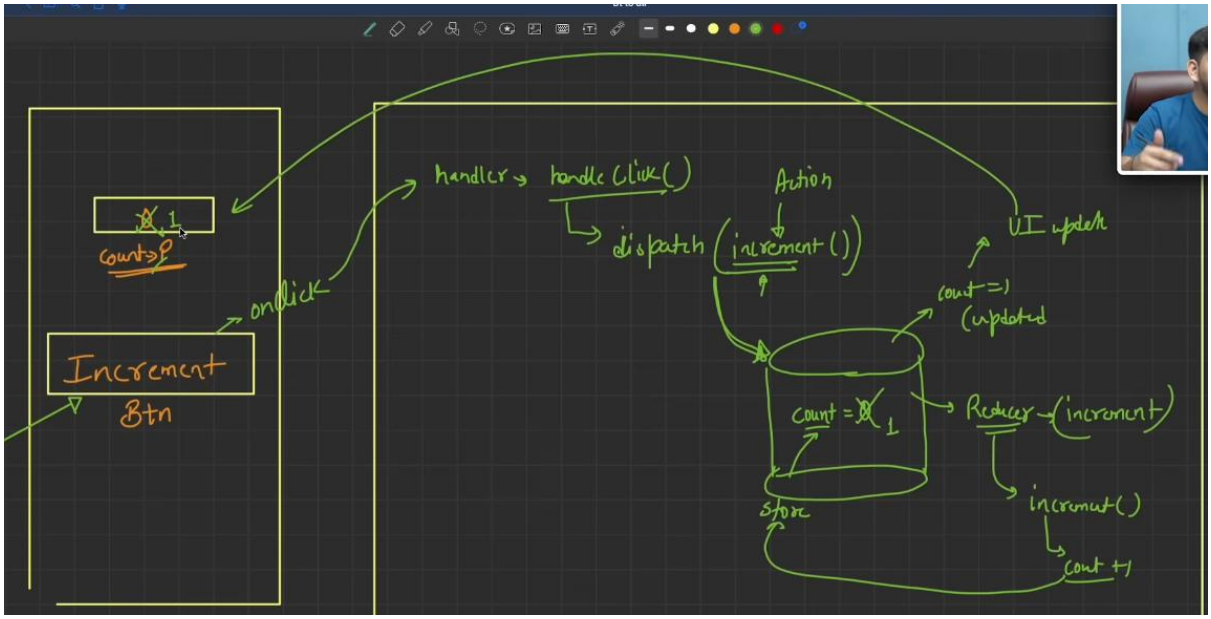
- A store is the central object that holds and manages the entire Redux state of your application.
- Component → Action → Reducer/Slice → Store → State → Component updates

- It basically holds the state, then receives the actions, and passes it to appropriate reducers (it passes currentState and action), it then update the store with the new value returned from the Reducers, and then it notifies react to re-render.
- A store is the central Redux container that holds the application's state, receives actions, runs reducers, and provides updated state to components.

Basically,

- **Action** → "What happened?"
- **Reducer** → "How should state change?"
- **State** → "Current data"
- **Slice** → "Feature-specific Redux logic" (Action + Reducer in RTK)
- **Store** → "The central place managing all state and reducers"

Example:



Redux data flow operates in a continuous, one-way cycle starting right from the user interface. It begins when a user clicks the "Increment Btn" on the UI, which originates an onClick event. This event triggers an event handler function called handleClick(). Inside this handler, the dispatch() function is called, sending a specific Action (in this case, the increment() action creator) toward the centralized Store. The Reducer function inside the store receives this action, processes the logic (count + 1), and uses it to perform the state transition. Once the reducer updates the state inside the Store (changing the count from 0 to 1), a UI Update is automatically triggered, pushing the newly updated state back to the frontend screen to change the displayed count value from 0 to 1.

How to install redux toolkit?

```
PS D:\ReactJS-revisited\learn-use-effect\React-useeffect-project> npm install @reduxjs/toolkit

added 7 packages, and audited 185 packages in 5s

40 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS D:\ReactJS-revisited\learn-use-effect\React-useeffect-project>
```

Also, install this as well:

```
PS D:\ReactJS-revisited\learn-use-effect\React-useeffect-project> npm install react-redux

added 3 packages, and audited 188 packages in 2s

40 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS D:\ReactJS-revisited\learn-use-effect\React-useeffect-project>
```

Now, let's see an example:

We will start by creating the store:

Store.js:

src > redux > JS store.js > store

```
1  import { configureStore } from "@reduxjs/toolkit";
2
3  export const store = configureStore({
4      reducer: {},
5  })
```

Now, wrapping the application using the Provider: we did it to make the store accessible across the entire application.

Main.jsx:

src > main.jsx

```
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import './index.css'
4  import App from './App.jsx'
5  // Importing the Provider and store
6  import { Provider } from 'react-redux'
7  import { store } from './redux/store.js'
8
9
10 createRoot(document.getElementById('root')).render(
11   // <StrictMode>
12   <Provider store={store}>
13     <App />
14   </Provider>
15   // </StrictMode>,
16 )
```

Now, we will create the slice: for this we created one folder in the src as features and inside it we created another folder named counter, and inside it we created the counterSlice.jsx.

CounterSlice.jsx:

src > features > CounterSlice.jsx > CounterSlice

```jsx
import { createSlice } from '@reduxjs/toolkit'
export const CounterSlice = createSlice({
  name: 'counter',
  initialState: {
    value: 0,
  },
  reducers: {
    increment: state => {
      state.value += 1
    },
    decrement: state => {
      state.value -= 1
    },
    incrementByAmount: (state, action) => {
      state.value += action.payload
    }
  }
})
// Action creators are generated for each case reducer function
export const { increment, decrement, incrementByAmount } = counterSlice.actions

export default CounterSlice.reducer
```

Now, we will register the counter to the store:

Store.js:

src > redux > JS store.js > store

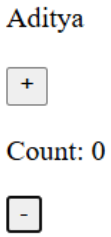
```jsx
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from '../features/counter/CounterSlice'
export const store = configureStore({
    reducer: {
        counter: counterReducer
    },
})
```

Now, we will create the UI.

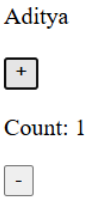
src > App.jsx > App

```jsx
import React from 'react'
import { useDispatch, useSelector } from 'react-redux'
import { increment, decrement } from './features/counter/CounterSlice';

function App() {
  const count = useSelector((state)=>state.counter.value)
  const dispatch = useDispatch();
  function handleIncrementClick(){
    dispatch(increment());
  }
  function handleDecrementClick(){
    dispatch(decrement());
  }
  return (
    <div>
      Aditya
      <br/>
      <br/>
      <button onClick={handleIncrementClick}>
        +
      </button>
      <p>Count: {count}</p>
      <button onClick={handleDecrementClick}>
        -
      </button>
    </div>
  )
}

export default App
```

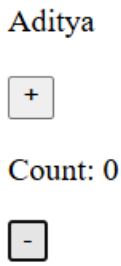
Output:



Output: when + was clicked.



Output: when - was clicked.



State Lifting in React:

What is State lifting?

- State lifting is a React pattern where state is moved from a child component to the closest common parent component so that multiple components can share and use the same data.
- State lifting in React is the process of moving state from a child component to its nearest common parent so that multiple components can share and stay synchronized with the same data. It creates a single source of truth and allows data to be passed down through props.

When Should You Lift State Up?

Lift state up when:

- Multiple components need the same data.
- One component updates data that another component displays.
- You want a single source of truth for the data.

Examples:

- Shopping cart count shown in multiple places.
- Search input and search results.
- Form fields whose values affect other components.
- Theme (dark/light mode) shared across components.

Now, let's first see an example of passing data from parent to child, which is the normal thing.

App.jsx:

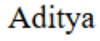
src > App.jsx > default

```jsx
import React from 'react'
import Card from './components/Card'

function App() {
  return (
    <div>
      <Card name="Aditya"/>
    </div>
  )
}

export default App
```

Card.jsx:

```jsx
src > components > Card.jsx > ...
import React from 'react'

function Card(props) {
  return (
    <div>
      {props.name}
    </div>
  )
}

export default Card
```

Output:



Now, to make this example as child to parent communication, we will make sure that we will be doing these things in the parent:

- ➔ Create state
- ➔ Manage state
- ➔ Change state
- ➔ Syncing state in all child

But how will we do this?

App.jsx:

```jsx
src > App.jsx > ...
import React, { useState } from 'react'
import Card from './components/Card'

function App() {
  const [name, setName] = useState('');
  return (
    <div>
      <Card name="Aditya" setName={setName}/>
      <p>I am inside the parent component and vlaue of name is: {name}</p>
    </div>
  )
}

export default App
```

App.jsx is the parent component that stores and manages the name state using useState. It passes the setName function to the Card component, allowing the child to update the parent's state whenever the user types in the input field. Whenever name changes, App re-renders and displays the latest value inside the <p> tag. In simple terms, App owns the data (name) and lets Card update it, which is an example of lifting state up in React.

Card.jsx:

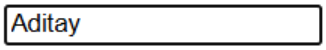
```
src > components > Card.jsx > default
1   import React from 'react'
2
3   function Card(props) {
4     return (
5       <div>
6         <input type="text" onChange={(e) => props.setName(e.target.value)}/>
7       </div>
8     )
9   }
10
11  export default Card
```

Card.jsx is a child component that displays a text input box. It doesn't store any data itself. Instead, whenever the user types something, it captures the input value and calls props.setName(e.target.value), which updates the name state stored in the parent component (App). In simple terms, Card collects the user's input and sends it to the parent to be saved and managed.

Output:

I am inside the parent component and vlaue of name is:

Output: when we start typing.



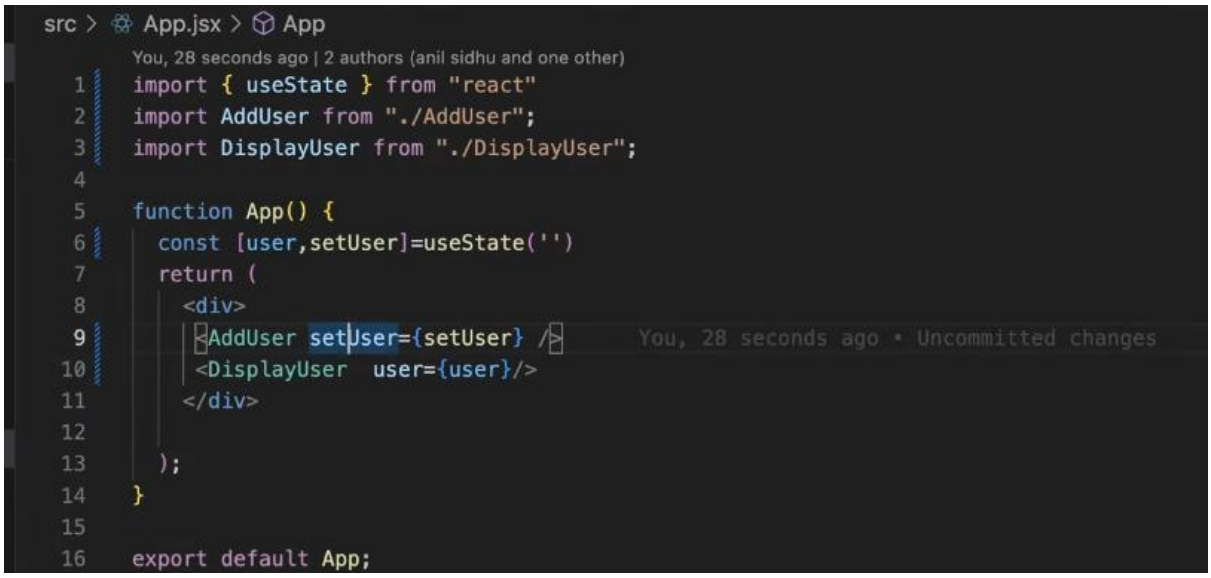
I am inside the parent component and vlaue of name is: Aditay

How is this state lifting?

The state that is related to the input could have been kept inside Card, but instead it is "lifted up" and stored in App. The child (Card) updates the state, while the parent (App) owns and manages it. This process of moving state from a child to its parent is called state lifting (lifting state up).

Now, we can even pass the data among siblings, which is also an example of state lifting.

App.jsx:



DispalyUser.jsx:

AddUser.jsx:

Output: